A Self-Hosted Platform for Programming Assignments

Andreas Auer

28th August 2026

OUTLINE

1. Problem Definition & Requirements
2. Technical Background
3. Architecture Overview
4. Demo
5. Integration Tests
6. Conclusion & Limitations
7. Future Work

Problem Definition & Requirements

Setting

University programming courses with many students, repeated assignments, and a self-hosted GitLab as the system of record.

Goal

One workflow to create, distribute, solve, test, and grade programming tasks — in the browser or locally, against the same automated tests.

> Target: A hand-in platform that runs untrusted student code safely, automates the assignment lifecycle, and stays low-maintenance for the teaching staff.

Usable

- Browser editor *or* local Git.
- One workflow for the assignment lifecycle.

Secure

- Untrusted student code runs isolated from the host.
- Roles, rate limits, and privileged actions are enforced server-side.

Usable

- Browser editor *or* local Git.
- One workflow for the assignment lifecycle.

Secure

- Untrusted student code runs isolated from the host.
- Roles, rate limits, and privileged actions are enforced server-side.

Self-hosted

- Runs on university infrastructure only.
- No third-party SaaS.

Low-maintenance

- Automated assignment lifecycle and grading.
- Integration tests against the deployed stack.
- No database.

Technical Background

TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.

TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.
GitLab CI pipeline	Ordered jobs started by GitLab after a push, button click, or API trigger.

TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.
GitLab CI pipeline	Ordered jobs started by GitLab after a push, button click, or API trigger.
GitLab runner	Daemon that asks GitLab for jobs and starts the matching job containers.

TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.
GitLab CI pipeline	Ordered jobs started by GitLab after a push, button click, or API trigger.
GitLab runner	Daemon that asks GitLab for jobs and starts the matching job containers.
Ansible	Deployment automation that installs hosts, renders configuration, and keeps secrets consistent.

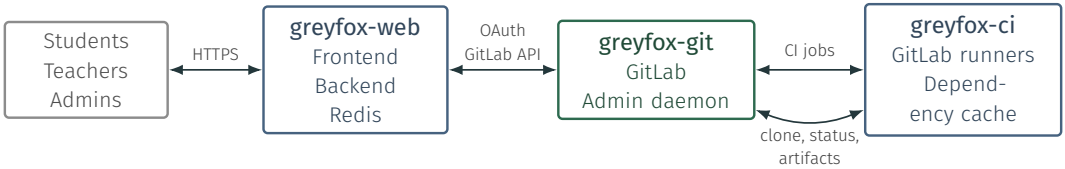
TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.
GitLab CI pipeline	Ordered jobs started by GitLab after a push, button click, or API trigger.
GitLab runner	Daemon that asks GitLab for jobs and starts the matching job containers.
Ansible	Deployment automation that installs hosts, renders configuration, and keeps secrets consistent.
Redis	In-memory store for sessions, rate limits, and short-lived platform state.

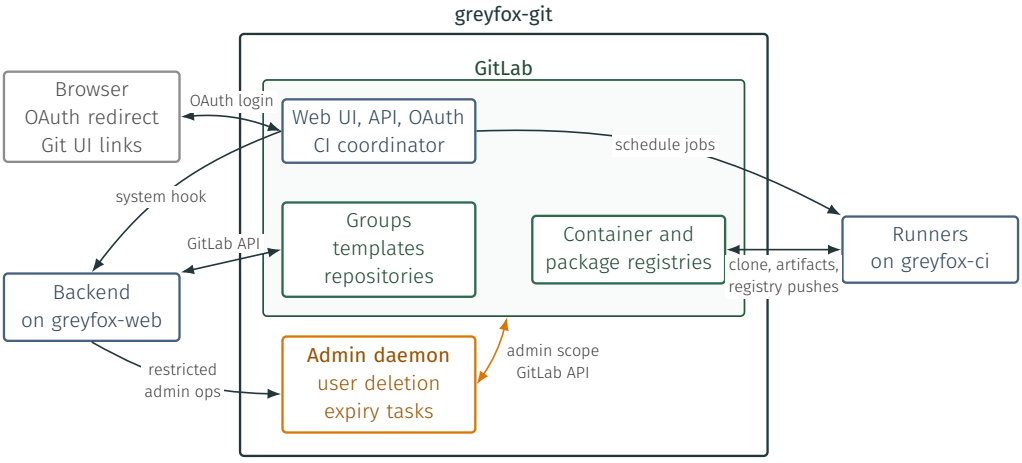
TERMS USED IN THE ARCHITECTURE

Term	Meaning
Docker	Runs each service or CI job in an isolated container with a defined image.
GitLab CI pipeline	Ordered jobs started by GitLab after a push, button click, or API trigger.
GitLab runner	Daemon that asks GitLab for jobs and starts the matching job containers.
Ansible	Deployment automation that installs hosts, renders configuration, and keeps secrets consistent.
Redis	In-memory store for sessions, rate limits, and short-lived platform state.
Caddy	Reverse proxy that terminates HTTPS and routes browser traffic to the frontend or backend.

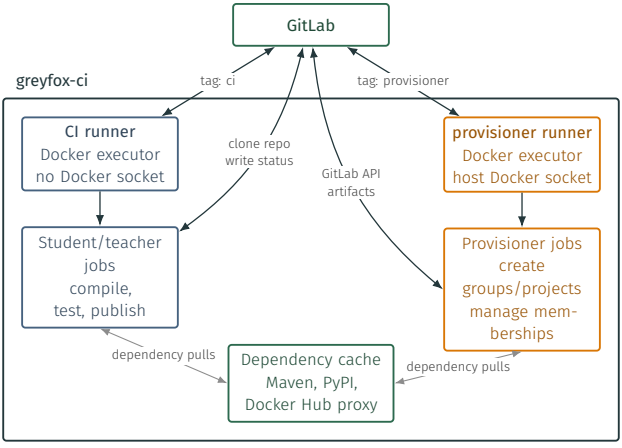
Architecture Overview

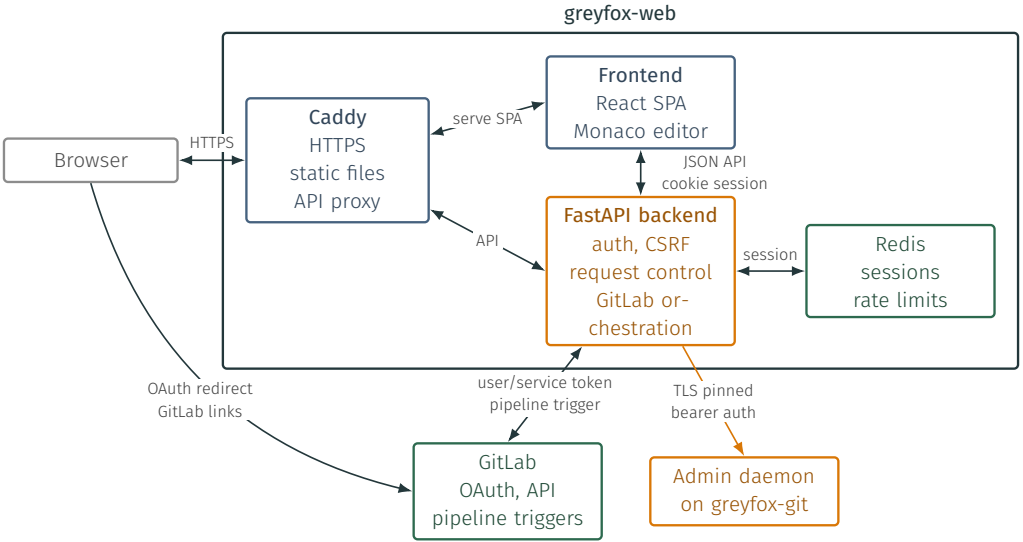


Core split: web validates requests and holds service secrets; GitLab stores users, repositories, CI metadata and registries; CI executes the untrusted and privileged jobs.


greyfox-git
GitLab
Browser
OAuth redirect
Git UI links
OAuth login
Web UI, API, OAuth
CI coordinator
schedule jobs
system hook
Groups
templates
repositories
GitLab API
Container and
package registries
Runners
on greyfox-ci
clone, artifacts,
registry pushes
Backend
on greyfox-web
restricted
admin ops
Admin daemon
user deletion
expiry tasks
admin scope
GitLab API

CI SERVER


GitLab
greyfox-ci
tag: ci
tag: provisioner
CI runner
Docker executor
no Docker socket
clone repo
write status
GitLab API
artifacts
provisioner runner
Docker executor
host Docker socket
Student/teacher
jobs
compile,
test, publish
Provisioner jobs
create
groups/projects
manage mem-
berships
dependency pulls
Dependency cache
Maven, PyPI,
Docker Hub proxy
dependency pulls


greyfox-web
Frontend
React SPA
Monaco editor
Caddy
HTTPS
static files
API proxy
serve SPA
Browser
HTTPS
JSON API
cookie session
FastAPI backend
auth, CSRF
request control
GitLab or-
chestration
API
session
Redis
sessions
rate limits
OAuth redirect
GitLab links
user/service token
pipeline trigger
TLS pinned
bearer auth
GitLab
OAuth, API
pipeline triggers
Admin daemon
on greyfox-git

Demo

- Admin: creates the department/course structure and adds teaching staff.

- Admin: creates the department/course structure and adds teaching staff.
- Teacher: selects template, publishes tests, and triggers provisioning.
- Teacher: creates the assignment and shares the invitation with students.

- Admin: creates the department/course structure and adds teaching staff.
- Teacher: selects template, publishes tests, and triggers provisioning.
- Teacher: creates the assignment and shares the invitation with students.
- Student: claims the invitation and receives a personal repository.
- Student: solves in the browser editor or locally with Git.
- Student: runs compile/test jobs and receives feedback.

- Admin: creates the department/course structure and adds teaching staff.
- Teacher: selects template, publishes tests, and triggers provisioning.
- Teacher: creates the assignment and shares the invitation with students.
- Student: claims the invitation and receives a personal repository.
- Student: solves in the browser editor or locally with Git.
- Student: runs compile/test jobs and receives feedback.
- Teacher: reviews status and submitted solutions.

Integration Tests

Testing beyond unit tests

Cross-component contracts: tokens shared between hosts, runner registration tags, GitLab system hook signing, env rendered by Ansible from bootstrap state, the backend–provisioner pipeline contract.

What this test does

Drives the real deployed stack end-to-end — the real GitLab API, real CI runners, and real provisioner pipelines — inside an isolated `e2e-<epoch>` namespace, then cleans up.

> Verifies that the deployed components actually agree on tokens, URLs, runner tags, repository paths, group variables, and API contracts.

Covered workflows

- OAuth login and logout,
- mother and child template creation,
- invitation claim and join flows,
- save, compile, test, publish,
- username policy system hook.

Covered workflows

- OAuth login and logout,
- mother and child template creation,
- invitation claim and join flows,
- save, compile, test, publish,
- username policy system hook.

Negative checks

- A missing CSRF token is rejected,
- duplicate requests do not create duplicates,
- outsider access is denied,
- the admin daemon is network-gated.

Covered workflows

- OAuth login and logout,
- mother and child template creation,
- invitation claim and join flows,
- save, compile, test, publish,
- username policy system hook.

Negative checks

- A missing CSRF token is rejected,
- duplicate requests do not create duplicates,
- outsider access is denied,
- the admin daemon is network-gated.

Trade Offs: Slower than unit tests, but it catches deployment regressions: stale bootstrap state, wrong runner registration, missing variables, broken OAuth redirects, or incompatible backend/provisioner changes.

Conclusion & Limitations

Conclusion

- Secure, reproducible, self-hosted hand-in system,
- stateless frontend and backend,
- supports Java and Python assignments,
- runners isolate untrusted student code,
- integration test guards against deployment drift.

Known limitations

- Pipeline speed: CI queueing, container startup, and job overhead dominate,
- teacher/admin flows span backend routes and provisioner CI jobs,
- no high availability: `greyfox-git` is a single point of failure,
- no tested backup or restore process yet.

Future Work

FUTURE WORK

- Replace provisioner pipeline with an admin service,
- fast compile service (possibly in-browser),
- reproducible disaster-recovery rebuild,
- ticket system,
- admin dashboard (e.g. system load),
- group finding feature.

Questions?